microgpt: Study Guide

Cheat sheets, a map from concepts to the literature, a self-test, and an annotated bibliography

Companion to the *microgpt* lecture notes and slides, based on Karpathy (2026b)

Compiled October 11, 2026

How to use this guide. Part I is a set of one-page cheat sheets, one per component of the program, meant to be read next to the code. Part II connects every idea in microgpt to the paper or book it comes from, and says what to read next. Part III is a self-test with answers. Part IV is the full bibliography, grouped by theme and annotated. Equation numbers and section names follow the lecture notes.

The one-sentence summary. A GPT is a function from a token sequence to a probability distribution over the next token, built from embeddings, attention (communication between positions) and MLPs (computation within a position) on a residual stream, trained by minimising cross-entropy with Adam using gradients from reverse-mode autodiff, and used by sampling one token at a time and feeding it back.

Contents

I Cheat sheets 3

1 The whole program on one page 3

2 Autograd cheat sheet 4

3 Architecture and attention cheat sheet 5

4 Training cheat sheet 6

5 Inference and decoding cheat sheet 7

6 microgpt, GPT-2 and a modern LLM side by side 8

7 Glossary 8

II From the code to the literature 9

8 Dataset and the language-modelling objective 9

9 Tokenizer 9

10 Autograd 9

11 Parameters and initialisation 9

12 Architecture 10

13 Training 10

14 Inference 10

15 Post-training (not in microgpt, discussed in the post) 11

16 Suggested reading order 11

III Self-test 12

17 Dataset and tokenizer 12

18 Autograd 12

19 Parameters 13

20 Architecture 13

21 Training 14

22 Inference 14

23 Scaling up 15

IV Annotated bibliography 16

Part I

Cheat sheets

1 The whole program on one page

Pipeline (section numbers as in the notes)

1. **Dataset**: 32,033 names, one per line = one document each.
2. **Tokenizer**: 26 letters + `BOS` $\Rightarrow V = 27$.
3. **Autograd**: `Value` scalars, 6 primitive ops, `backward()`.
4. **Parameters**: 4,192 floats $\sim \mathcal{N}(0, 0.08^2)$ in a `state_dict`.
5. **Architecture**: GPT-2 minus LayerNorm/biases/GeLU; 1 layer, 4 heads, $d = 16$.
6. **Training**: 1,000 steps, one document each, cross-entropy, Adam, linear decay.
7. **Inference**: sample from `BOS` at $T = 0.5$ until `BOS`; 20 names.

Hyperparameters

Name	Value	Meaning
`n_embd` (d)	16	width of the residual stream
`n_head` (H)	4	attention heads
`head_dim` (d_h)	4	d/H
`n_layer` (L)	1	Transformer blocks
`block_size` ($T_{\max}$)	16	max context (longest name 1
`vocab_size` (V)	27	tokens incl. `BOS`
init std	0.08	Gaussian weight scale
`learning_rate`	0.01	peak LR, linear decay to 0
$\beta_1, \beta_2, \epsilon$	0.85, 0.99, 10^{-8}	Adam
`num_steps`	1000	one document per step
`temperature`	0.5	sampling sharpness
seed	42	`random.seed`

Numbers to remember

- $\ln 27 = 3.296$: loss of a uniform guess. First printed loss 3.366.
- Final smoothed loss ≈ 2.3 (post: ≈ 2.37); last single-document loss 2.65.
- Perplexity $e^{\mathcal{L}}$: 27 $\rightarrow\approx$ 10. Bits/char: 4.75 $\rightarrow\approx$ 3.4.
- Model sees 1,000 of 32,033 names: 3% of an epoch; no overfitting.
- Graph size per step: $\approx$ 8k `Value` nodes per token ($\approx 10^5$ for a 12-letter name).
- Runtime: about 1 min (Karpathy) to 3.5 min (our run) in CPython.
- GPT-2: 1.5B params; GPT-3: 175B; microgpt: 4,192.

Shapes (column-vector convention, $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$)

Object	Shape	Lives in
`wte`	$V \times d = 27 \times 16$	params
`wpe`	$T_{\max} \times d = 16 \times 16$	params
W_q, W_k, W_v, W_o	$d \times d = 16 \times 16$	params (per layer)
W_1 (`mlp_fc1`)	$4d \times d = 64 \times 16$	params
W_2 (`mlp_fc2`)	$d \times 4d = 16 \times 64$	params
`lm_head`	$V \times d = 27 \times 16$	params
x (residual stream)	$d = 16$	activation
q, k, v	16, sliced into 4×4	activation
cache `keys[l]`	list of i+1 vectors of 16	activation
attention weights (head h)	i+1	activation
MLP hidden	64	activation
logits	27	activation

Parameter count

$$N = \underbrace{Vd}_{\text{wte}} + \underbrace{T_{\max} d}_{\text{wpe}} + L\Big(\underbrace{4d^2}_{\text{attn}} + \underbrace{8d^2}_{\text{MLP}}\Big) + \underbrace{Vd}_{\text{lm_head}}$$

$= 432 + 256 + 1 \cdot (1024 + 2048) + 432 = 4{,}192$. Per block $12d^2$.

Design choices vs. GPT-2

	GPT-2	microgpt
Normalisation	LayerNorm (gain, bias)	RMSNorm, no gain
Biases	yes	none
Activation	GELU	ReLU
Position	learned table	learned table
Output head	tied to `wte`	separate
Norm placement	pre-norm	pre-norm
Dropout	yes	none

Three things people get wrong. (1) Token ids are symbols, not magnitudes. (2) There is no causal mask because the cache contains only the past. (3) The printed loss is per document, so it is noisy; judge progress with a running mean.

2 Autograd cheat sheet

The six primitives (forward value, stored local gradient)

Op	Forward	Local grads
`a + b`	$a + b$	$(1,\ 1)$
`a * b`	ab	$(b,\ a)$
`a ** n`	a^n	na^{n-1}
`a.log()`	$\ln a$	$1/a$
`a.exp()`	e^a	e^a
`a.relu()`	$\max(0, a)$	$\mathbf{1}[a > 0]$

Derived: `-a = a*-1`, `a-b = a+(-b)`, `a/b = a*b**-1`; the `__r*__` variants let plain floats appear on the left.

A node

`Value(data, children, local_grads)`: `.data` (forward scalar), `.grad` (accumulated $\partial \mathcal{L}/\partial v$, starts at 0), `_children` (inputs), `_local_grads` ($\partial v/\partial$child, computed at construction).

`backward()` in three lines

1. DFS from the root; append a node after its children $\Rightarrow$ topological order.
2. Seed: `self.grad = 1` ($\partial \mathcal{L}/\partial \mathcal{L} = 1$).
3. For v in reverse order, for each child c: `c.grad += local * v.grad`.

$$\frac{\partial \mathcal{L}}{\partial c} \mathrel{+}= \frac{\partial v}{\partial c} \cdot \frac{\partial \mathcal{L}}{\partial v}$$

Worked example

$L = ab + a$, $a = 2$, $b = 3$: $\partial L/\partial a = b + 1 = 4$ (two paths), $\partial L/\partial b = a = 2$. PyTorch agrees.

Why reverse mode

One scalar output, many inputs: one backward sweep costs about as much as one forward pass and yields *all* partial derivatives. Forward-mode would need one sweep per input. (Baydin et al. 2018; Griewank and Walther 2008)

Gotchas

- **+=, not =.** A value used twice gets gradient from both uses (multivariable chain rule).
- **Zero the grads** after each update, or they accumulate across steps.
- **Order matters.** A node's gradient must be complete before it is pushed to its children; topological order guarantees this.
- **Constants.** `max_val` in softmax is a float, not a `Value`: the shift is constant in the graph.
- **Graph per step.** The graph is rebuilt every forward pass (define-by-run), exactly like PyTorch.
- **Cost.** One node per scalar op: $\approx$ 8k nodes per token here.

Gradient of softmax + cross-entropy

With $p = \text{softmax}(z)$ and target y one-hot: $\partial \ell/\partial z_k = p_k - y_k$. microgpt never writes this; the engine derives it from `exp`, `log`, sum and division nodes.

Lineage

Reverse mode: Linnainmaa (1976). Backprop for neural nets: Rumelhart et al. (1986). PyTorch autograd: Paszke et al. (2017). micrograd: Karpathy (2020, 2022d).

3 Architecture and attention cheat sheet

One forward call, $x \in \mathbb{R}^{16}$

$$x \leftarrow \text{rmsnorm}(\text{wte}[t_i] + \text{wpe}[i])$$

for each layer:

$$x \leftarrow x + W_o\,\text{MHA}(\text{rmsnorm}(x))$$
$$x \leftarrow x + W_2\,\text{relu}(W_1\,\text{rmsnorm}(x))$$
$$z = W_{\text{out}}\,x \in \mathbb{R}^{27}$$

Softmax of z is done by the caller (loss or sampler).

Helpers

- `linear(x, w)`: $y_j = \sum_i w_{ji} x_i$ (one dot product per row).
- `softmax(z)`: $e^{z_k - m} / \sum_j e^{z_j - m}$, $m = \max z$ for stability.
- `rmsnorm(x)`: $x / \sqrt{\frac{1}{d}\sum x_i^2 + 10^{-5}}$; no mean subtraction, no gain.

Attention, head h, position i

$$q = W_q x,\ k = W_k x,\ v = W_v x \quad \text{(append } k, v \text{ to cache)}$$
$$a_s = q[S_h] \cdot k_s[S_h] / \sqrt{d_h}, \qquad s = 0..i$$
$$\alpha = \text{softmax}(a_0..a_i)$$
$$\text{head}_h = \textstyle\sum_s \alpha_s\, v_s[S_h] \in \mathbb{R}^{d_h}$$
$$\text{out} = W_o\,[\text{head}_1 \| \ldots \| \text{head}_H]$$

S_h = coordinates $hd_h..(h{+}1)d_h{-}1$. Matrix form: $\text{softmax}(QK^\top/\sqrt{d_h})V$.

Mnemonics

Query = "what am I looking for?" Key = "what do I contain?" Value = "what do I offer if selected?" Attention = *communication* across positions; MLP = *computation* within a position.

Why each piece

- **Position embedding**: the only source of order information (attention is a weighted sum). Learned here; RoPE in modern models.
- $1/\sqrt{d_h}$: keeps dot-product variance at 1 so softmax is not saturated at init.
- **Multiple heads**: same cost as one wide head; buys different lookup patterns.
- **Residual**: identity path for gradients; depth becomes trainable.
- **Pre-norm**: normalise the branch input, not the stream; stabler than post-norm.
- **First rmsnorm**: sets the scale at which the residual stream starts; "not redundant".
- **MLP** 4×: GPT-2 convention; most parameters live here ($8d^2$ vs $4d^2$).

KV cache facts

- Per layer: lists `keys[l]`, `values[l]` of the k, v for positions $0..i$.
- Causality is automatic: nothing from the future is in the cache. No mask.
- During training the entries are live `Values`: gradients flow through the cache.
- Size $2\,L\,T\,d$ per sequence; at scale this drives MQA/GQA and paged caches.

Cost per token

$\approx 12d^2 + Vd + 2(i{+}1)d$ multiply–adds $\approx$ 3.5k here; the "$2N$ forward, $6N$ train" rule of scaling-law papers.

Lineage

Attention: Bahdanau et al. (2015) and Vaswani et al. (2017). GPT-2: Radford et al. (2019). RMSNorm: Zhang and Sennrich (2019). Residuals: He et al. (2016). Pre-norm: Xiong et al. (2020). Residual stream: Elhage et al. (2021). Pseudocode for all variants: Phuong and Hutter (2022).

4 Training cheat sheet

One step

1. `doc = docs[step % len(docs)]`; tokens = `[BOS]` + chars + `[BOS]`; $n = \min(16, |\text{tokens}| - 1)$.
2. Empty cache. For $i = 0..n-1$: logits = `gpt(tokens[i], i, keys, values)`; $\ell_i = -\log \text{softmax}(z)_{t_{i+1}}$.
3. $\mathcal{L} = \frac{1}{n}\sum_i \ell_i$; `loss.backward()`.
4. Adam update with $\eta_t = 0.01(1 - t/1000)$; zero grads.

Cross-entropy, three readings

$\ell = -\log p(\text{target})$ is (a) the model's *surprise*; (b) a term of the negative log-likelihood of the corpus; (c) the cross-entropy $H(y, p) = H(y) + \text{KL}(y\|p)$ with a one-hot y.

Reference values

	loss (nats)	perplexity
uniform over 27	3.296	27
step 1	3.366	29
smoothed, step 1000	≈ 2.3	≈ 10
perfect	0	1

Bits per character = $\mathcal{L}/\ln 2$.

Adam

$$m \leftarrow \beta_1 m + (1-\beta_1)g, \qquad \hat{m} = m/(1-\beta_1^t)$$
$$v \leftarrow \beta_2 v + (1-\beta_2)g^2, \qquad \hat{v} = v/(1-\beta_2^t)$$
$$\theta \leftarrow \theta - \eta_t\, \hat{m}/(\sqrt{\hat{v}} + \epsilon)$$

m = momentum; v = per-parameter scale (RMSProp); hats = bias correction for zero init. t = step + 1.

Batch size one

- Each step is one document $\Rightarrow$ the loss is noisy: short common names score low, long rare ones high.
- Judge progress by a trailing mean (50 steps: ≈ 2.3 at the end of our run).
- 1,000 steps see 1,000 distinct names (shuffled): every step is on fresh data, so the training loss is effectively a held-out loss.

Knobs and their effect

Knob	Effect
`num_steps`↑	lower loss until data repeats
`n_embd`, `n_layer`↑	more capacity; slower
`learning_rate`↑	faster, then unstable
β_1 ↑	smoother, laggier momentum
β_2 ↑	longer memory of gradient scale
batch $B > 1$	less noise per step; $B\times$ cost

What production adds

Warmup then cosine/linear decay; AdamW weight decay; batches of 10^6 tokens; gradient accumulation; bf16/fp8; scaling laws to choose N and tokens (≈ 20 tokens per parameter, Chinchilla).

Lineage

SGD: Robbins and Monro (1951). Momentum: Polyak (1964). RMSProp: Tieleman and Hinton (2012). Adam: Kingma and Ba (2015). AdamW: Loshchilov and Hutter (2019). Schedules: Loshchilov and Hutter (2017). Scaling: Hoffmann et al. (2022) and Kaplan et al. (2020). Cross-entropy and information: Cover and Thomas (2006) and Shannon (1948).

5 Inference and decoding cheat sheet

Sampling loop

1. Empty cache; `token_id = BOS`.
2. For $i = 0..T_{\max}-1$: logits = `gpt(token_id, i, keys, values)`.
3. $p = \text{softmax}(z/T)$; draw `token_id` $\sim p$.
4. Stop if `BOS`; else append `uchars[token_id]` and continue.

Parameters are frozen; the graph is still built but never backpropagated.

Temperature

$p_k^{(T)} \propto e^{z_k/T}$. $T = 1$: model's distribution. $T < 1$: sharper (ratio of top two probabilities raised to $1/T$). $T \to 0$: greedy. $T > 1$: flatter.

Other decoders (not in microgpt)

- **Greedy**: $\arg\max_k z_k$.
- **Top-k**: zero out all but the k largest probabilities, renormalise (Fan et al. 2018).
- **Nucleus / top-p**: keep the smallest set whose mass $\geq p$ (Holtzman et al. 2020).
- **Beam search**: keep B best prefixes; common in translation, rare in chat.

The default samples (seed 42)

kamon, ann, karai, jaire, vialan, karia, yeran, anna, areli, kaina, konna, keylen, liole, alerin, earan, lenne, kana, lara, alela, anton.

"Hallucination" in one line

The model samples statistically plausible sequences; it has no representation of truth. A made-up name and a made-up fact are the same mechanism.

Production serving

Continuous batching (Yu et al. 2022); paged KV cache (Kwon et al. 2023); speculative decoding (draft model proposes, target verifies) (Chen et al. 2023; Leviathan et al. 2023); int8/int4 weights (Dettmers et al. 2022; Frantar et al. 2023); tensor/pipeline parallelism across GPUs (Pope et al. 2022).

Lineage

Temperature: Ackley et al. (1985) and Hinton et al. (2015). Degeneration and nucleus sampling: Holtzman et al. (2020).

6 microgpt, GPT-2 and a modern LLM side by side

	microgpt (2026)	**GPT-2 (2019)**	**Modern open LLM (LLaMA-style, 2023–)**
Parameters	4,192	124M – 1.5B	10^9 – 10^{12} (dense or MoE)
Width / depth / heads	16 / 1 / 4	768–1600 / 12–48 / 12–25	4k–16k / 32–128 / 32–128
Context	16 tokens	1,024	10^5 – 10^6
Tokenizer	27 characters	byte-level BPE, 50,257	BPE / SentencePiece, 32k–256k
Position	learned table	learned table	RoPE
Normalisation	RMSNorm, no gain	LayerNorm	RMSNorm with gain
Attention	MHA, explicit cache	MHA, masked matmul	GQA / MQA / MLA, FlashAttention
MLP	4×, ReLU	4×, GELU	SwiGLU (≈ 2.7×), or MoE experts
Biases	none	yes	none
Output head	separate	tied to `wte`	tied or separate
Optimizer	Adam, linear decay	Adam	AdamW, warmup + cosine, weight decay
Batch	1 document	512 sequences	10^6 – 10^7 tokens
Precision	float64 (Python)	float32	bf16 / fp8
Training data	32k names (≈ 200k chars)	40 GB WebText	10^{13} tokens, curated
Post-training	none	none	SFT + RLHF / DPO / GRPO
Serving	Python loop	single process	vLLM-style batching, paging, speculation, quantisation

Sources: Ainslie et al. (2023), DeepSeek-AI (2024), Kwon et al. (2023), Radford et al. (2019), Shazeer (2020), Su et al. (2021), and Touvron et al. (2023). The exact numbers for modern models vary by release; the *pattern* of substitutions is what matters.

7 Glossary

Autograd Automatic computation of gradients by recording operations in a graph and applying the chain rule in reverse.
Autoregressive Generating a sequence one element at a time, each conditioned on the previous ones.
Bias correction Adam's division by $1 - \beta^t$ that compensates for zero-initialised moment buffers.
BOS Beginning-of-sequence token; here also the end-of-sequence marker.
BPE Byte-pair encoding: a subword tokenizer built by merging frequent adjacent symbol pairs.
Causal attention Attention restricted to current and earlier positions.
Chain rule $\partial L/\partial c = \sum_{\text{paths}} \partial L/\partial v \cdot \partial v/\partial c$.
Context length Maximum number of positions the model can attend over; `block_size`.
Cross-entropy $-\sum_k y_k \log p_k$; equals $-\log p_{\text{target}}$ for one-hot targets.
Document One training sequence; a name here, a web page in production.
Embedding A learned vector associated with a discrete symbol (token or position).
Gradient Vector of partial derivatives of the loss with respect to the parameters.
Hallucination A sampled output that is plausible under the model but not true or not real.
Head One of H parallel attention computations on a slice of the coordinates.
KV cache Stored keys and values of earlier positions, reused by later positions.
Logits Unnormalised scores before softmax.
LayerNorm / RMSNorm Normalisations that rescale a vector to unit variance / unit RMS.
MLP Two-layer feed-forward network applied independently at each position.
Momentum Exponential moving average of gradients used to smooth updates.
Next-token prediction The training objective: predict t_{i+1} from $t_{\le i}$.
Perplexity e^{loss}; effective number of equally likely choices.
Pre-norm Normalising the input of a residual branch rather than its output.
Residual connection $x \leftarrow x + f(x)$; the identity path through a block.
Residual stream The vector x that blocks read from and write to.
SFT Supervised fine-tuning on curated conversations.
RLHF Reinforcement learning from human feedback; post-training from scored samples.
Softmax Maps real scores to a probability distribution.
Temperature Divisor of the logits before softmax; controls sampling sharpness.
Token A discrete symbol from the vocabulary; a character here.
Topological order An ordering of a DAG in which every node precedes its dependents.
Vocabulary The set of tokens; $V = 27$.

Part II

From the code to the literature

Each entry below names an idea that appears in microgpt, the lines of code where it lives, the work that introduced or best explains it, and what to read next. The aim is that every line of the program can be traced to a source.

8 Dataset and the language-modelling objective

Text as a predictable sequence. The idea that language has measurable statistical structure, and that predicting the next symbol is a well-posed task, is Shannon's. Shannon (1948) defines entropy and n-gram approximations of English; Shannon (1951) measures how well humans predict the next letter, which is the character-level task microgpt solves. A modern textbook treatment of n-gram and neural language models is Jurafsky and Martin (2025).

Neural language models. Bengio et al. (2003) introduced the recipe microgpt still follows: embed each token into a learned vector, combine the vectors with a neural network, output a softmax over the vocabulary, train by maximum likelihood. Recurrent versions (Elman 1990; Hochreiter and Schmidhuber 1997; Mikolov et al. 2010) preceded Transformers; character-level generation with RNNs (Graves 2013; Karpathy 2015; Sutskever et al. 2011) is the direct ancestor of the names task, and the makemore project (Karpathy 2022a) is where `names.txt` comes from.

A conversation is a document. The post's remark that ChatGPT sees your chat as "a funny looking document" is the premise of GPT-1 and GPT-2 (Radford et al. 2018, 2019): one generic objective, next-token prediction, on diverse text. Scaling that objective produced few-shot learning (Brown et al. 2020). Data curation at scale: Gao et al. (2020), Lee et al. (2022), Penedo et al. (2024), and Raffel et al. (2020).

9 Tokenizer

Character tokens, `uchars`, `BOS`. The simplest possible tokenizer. The one design decision, a single special token used both to start and end a document, is a simplification of the `<|endoftext|>` delimiter in GPT-2.

Subword tokenization. BPE originates as a compression scheme (Gage 1994), was adapted for open-vocabulary machine translation by Sennrich et al. (2016), and became byte-level in GPT-2 (Radford et al. 2019); tiktoken is the production implementation (OpenAI 2022). SentencePiece (Kudo and Richardson 2018) is the language-independent alternative. Karpathy (2024a) builds a BPE tokenizer step by step.

10 Autograd

`Value`, `_children`, `_local_grads`, `backward()`. Reverse-mode automatic differentiation. First published by Linnainmaa (1976); rediscovered as backpropagation for neural networks by Rumelhart et al. (1986). The definitive mathematical reference is Griewank and Walther (2008); for a machine-learning reader Baydin et al. (2018) explains forward vs. reverse mode, why reverse mode suits a scalar loss, and the define-by-run style that microgpt and PyTorch share (Paszke et al. 2017, 2019). The scalar engine itself is micrograd (Karpathy 2020), taught in Karpathy (2022d).

Topological sort. The DFS post-order in `build_topo` is the standard algorithm for ordering a DAG; any algorithms textbook covers it. Its role here is to guarantee that a node's gradient is complete before being pushed to its children.

11 Parameters and initialisation

`matrix(nout, nin, std=0.08)`. Small random initialisation breaks symmetry and keeps early activations in a well-behaved range. The dependence of the right scale on fan-in is analysed by Glorot and Bengio (2010); GPT-2 used $\sigma = 0.02$. The absence of biases follows LLaMA and most later models (Touvron et al. 2023). The decision not to tie `lm_head` to `wte` departs from GPT-2 and from Press and Wolf (2017), who showed tying helps small models; here it keeps the code easier to read.

12 Architecture

`linear`. The matrix–vector product is the elementary operation of every neural network since Rumelhart et al. (1986); see any textbook (Bishop 2006; Goodfellow et al. 2016; Prince 2023).

`softmax` with max subtraction. Standard numerically stable softmax; the Boltzmann form goes back to statistical physics and entered neural networks through Ackley et al. (1985).

`rmsnorm`. Zhang and Sennrich (2019) proposed RMSNorm as a cheaper LayerNorm (Ba et al. 2016) that drops mean-centering; it is now standard (Touvron et al. 2023). microgpt also drops the learned gain.

Embeddings `wte`, `wpe`. Learned token embeddings date to Bengio et al. (2003). Learned absolute position embeddings are from GPT-1/2 (Radford et al. 2018, 2019); the original Transformer used sinusoids (Vaswani et al. 2017); RoPE (Su et al. 2021) is the modern replacement and is a good first extension exercise.

Attention: q, k, v, scaled dot product, softmax, weighted sum. Attention as a soft lookup over a sequence is from Bahdanau et al. (2015); the scaled dot-product, multi-head form is from Vaswani et al. (2017), which also explains the $1/\sqrt{d_h}$ scaling. Decoder-only causal use is GPT (Radford et al. 2018). The post's slogan that attention is a "token communication mechanism" matches the residual-stream analysis of Elhage et al. (2021). Alammar (2018) is the best-known visual explanation; Phuong and Hutter (2022) give precise pseudocode.

KV cache. Implicit in every autoregressive Transformer; made explicit for serving in Shazeer (2019), which also introduced multi-query attention to shrink it, generalised by grouped-query attention (Ainslie et al. 2023). The memory and bandwidth accounting that makes the cache the bottleneck of inference is in Pope et al. (2022); paged management is vLLM (Kwon et al. 2023).

MLP with ReLU. ReLU: Glorot et al. (2011) and Nair and Hinton (2010). GELU, which GPT-2 used: Hendrycks and Gimpel (2016). Gated (SwiGLU) MLPs: Shazeer (2020). Squared ReLU: So et al. (2021). Mixture-of-Experts MLPs: DeepSeek-AI (2024), Fedus et al. (2022), and Shazeer et al. (2017).

Residual connections and pre-norm. Residual learning: He et al. (2016). Pre-norm placement and why it trains more stably than the original post-norm Transformer: Xiong et al. (2020).

13 Training

Cross-entropy / negative log-likelihood. Maximum likelihood for neural LMs: Bengio et al. (2003). Information-theoretic reading (surprisal, entropy, KL, perplexity): Cover and Thomas (2006) and Shannon (1948); textbook derivations in Bishop (2006) and Goodfellow et al. (2016).

Stochastic gradient descent with one document per step. Robbins and Monro (1951). The modern large-batch regime and why batch size interacts with learning rate are discussed in Kaplan et al. (2020).

Adam with bias correction. Kingma and Ba (2015), building on momentum (Polyak 1964) and RMSProp (Tieleman and Hinton 2012). Decoupled weight decay (AdamW): Loshchilov and Hutter (2019). Learning-rate schedules: Loshchilov and Hutter (2017).

Scaling. How loss depends on parameters, data and compute: Kaplan et al. (2020); compute-optimal allocation (Chinchilla): Hoffmann et al. (2022). Mixed precision: Micikevicius et al. (2018). Fused attention kernels: Dao (2024) and Dao et al. (2022).

14 Inference

Ancestral sampling with temperature. Temperature: Ackley et al. (1985) and Hinton et al. (2015). Why pure sampling and greedy decoding both fail for long text, and the nucleus fix: Holtzman et al. (2020); top-k: Fan et al. (2018).

Serving. Continuous batching (Yu et al. 2022), PagedAttention (Kwon et al. 2023), speculative decoding (Chen et al. 2023; Leviathan et al. 2023), quantisation (Dettmers et al. 2022; Frantar et al. 2023), multi-device inference (Pope et al. 2022).

15 Post-training (not in microgpt, discussed in the post)

SFT and RLHF as used for InstructGPT and ChatGPT: Ouyang et al. (2022), built on Christiano et al. (2017) and PPO (Schulman et al. 2017). Direct preference optimisation removes the separate reward model (Rafailov et al. 2023); GRPO (Shao et al. 2024) is the group-relative policy-gradient variant used in recent reasoning models. nanochat (Karpathy 2025) implements pretraining, SFT, RL and serving in one small codebase.

16 Suggested reading order

1. The post (Karpathy 2026b) and the gist, with these cheat sheets open.
2. Karpathy (2022d) if `backward()` is not yet obvious; Baydin et al. (2018) for the theory.
3. Vaswani et al. (2017) and Alammar (2018) for attention; then Phuong and Hutter (2022) to see every variant written out.
4. Radford et al. (2019) to see the architecture microgpt simplifies; Touvron et al. (2023) to see what replaced it.
5. Kingma and Ba (2015) and Loshchilov and Hutter (2019) for the optimizer.
6. Kaplan et al. (2020) and Hoffmann et al. (2022) for how the knobs behave at scale.
7. Ouyang et al. (2022) for post-training; Kwon et al. (2023) for serving.
8. Karpathy (2022b) and Karpathy (2025) to run the real thing.

Part III

Self-test

Answer first, then reveal. Questions are grouped by section of the notes.

17 Dataset and tokenizer

Q1. How many documents, and what is one document?

A. 32,033; one name.

Q2. What is the vocabulary and its size?

A. The 26 lowercase letters plus `BOS`; $V = 27$.

Q3. Why does the post say token ids "might as well be different emoji"?

A. The integers are arbitrary labels; nothing in the model uses their numeric value. The embedding table attaches meaning.

Q4. What two jobs does `BOS` do?

A. It is the first input of every document (so the model can predict the first letter) and the target after the last letter (so the model learns when to stop).

Q5. What is the longest name, and why does it matter?

A. 15 characters; with two `BOS` tokens that is 17 tokens and 16 predictions, so `block_size = 16` never truncates.

18 Autograd

Q6. What four fields does a `Value` hold?

A. `data`, `grad`, `_children`, `_local_grads`.

Q7. When is the local gradient of `a * b` computed, and what is it?

A. At construction, during the forward pass: (b, a).

Q8. Why `+=` in `backward()`?

A. A node used in several places contributes to the loss along several paths; the multivariable chain rule sums them.

Q9. Why reverse topological order?

A. A node's gradient must be complete before it is multiplied into its children's gradients.

Q10. For $L = ab + a$ with $a = 2$, $b = 3$, what are the gradients?

A. $\partial L/\partial a = 4$, $\partial L/\partial b = 2$.

Q11. What is the difference between microgpt's autograd and PyTorch's?

A. Nodes are scalars instead of tensors; the algorithm is the same.

Q12. Why must gradients be zeroed after each update?

A. Because `backward()` accumulates with `+=`; stale gradients would be added to new ones.

19 Parameters

Q13. Compute the parameter count from the hyperparameters.

A. $27{\cdot}16 + 16{\cdot}16 + 4{\cdot}16^2 + 2{\cdot}64{\cdot}16 + 27{\cdot}16 = 432 + 256 + 1024 + 2048 + 432 = 4192$.

Q14. How many parameters per Transformer block, as a function of d?

A. $12d^2$: $4d^2$ in attention, $8d^2$ in the MLP.

Q15. Name three GPT-2 features microgpt omits.

A. Biases, LayerNorm (replaced by RMSNorm), GELU (replaced by ReLU); also weight tying and dropout.

20 Architecture

Q16. What are the inputs and output of `gpt()`?

A. A token id, a position id, the per-layer lists of cached keys and values (which it extends), and the parameters via globals; it returns 27 logits.

Q17. Why subtract the max in softmax?

A. To prevent overflow in `exp`; the result is unchanged mathematically.

Q18. What does `rmsnorm` compute and how does it differ from LayerNorm?

A. $x/\sqrt{\text{mean}(x^2) + \varepsilon}$; no mean subtraction and (here) no learned gain.

Q19. Where is the causal mask?

A. There is none: at position i the cache only contains positions $\leq i$.

Q20. Why divide attention logits by $\sqrt{d_h}$?

A. So dot products of unit-variance vectors have unit variance and the softmax is not saturated.

Q21. What is the only place where position i reads information from earlier positions?

A. The attention block; the MLP, norms and residual adds are per-position.

Q22. Do four heads of width 4 cost more than one head of width 16?

A. No; the projections have the same size. Heads buy diversity of lookups.

Q23. Why is the KV cache used during training here?

A. Because tokens are processed one at a time; the cache holds live `Values` so gradients flow through it. Vectorised code hides the same quantities in a matrix.

Q24. Why is the first `rmsnorm` (right after the embeddings) not redundant?

A. The pre-norm in the block normalises only the branch input; the residual path keeps the raw embedding scale, so an initial norm fixes the scale of the stream.

Q25. What is the MLP's hidden width and activation?

A. $4d = 64$; ReLU.

Q26. Roughly how many multiply–adds per token?

A. About $12d^2 + Vd \approx 3.5$k, about $2\times$ the non-embedding parameters.

21 Training

Q27. Write the per-position loss and the document loss.

A. $\ell_i = -\log p(t_{i+1} \mid t_{\le i})$; $\mathcal{L} = \frac{1}{n}\sum_i \ell_i$.

Q28. What loss does a uniform predictor get, and why?

A. $-\log(1/27) = \ln 27 \approx 3.30$.

Q29. The loss ends near 2.3. What perplexity is that?

A. $e^{2.3} \approx 10$: as uncertain as a uniform choice among ten letters.

Q30. Why is the printed loss so jumpy?

A. Each step is one document; short common names are easy, long rare ones hard.

Q31. Does the model overfit in 1,000 steps?

A. No; it sees 1,000 distinct names out of 32,033, so every step is on new data.

Q32. What do m and v track in Adam?

A. Exponential moving averages of the gradient and of the squared gradient.

Q33. What do the bias corrections fix?

A. m and v start at zero and are biased toward zero early on; dividing by $1 - \beta^t$ removes the bias.

Q34. What is the learning rate at step 500 and at the last step?

A. 0.005 and 10^{-5} (linear decay $0.01(1 - t/1000)$).

Q35. What are microgpt's Adam betas, and the paper defaults?

A. 0.85, 0.99 versus 0.9, 0.999.

22 Inference

Q36. Describe one sampling step.

A. Run `gpt` on the current token and position, divide logits by T, softmax, draw a token, append it, feed it back; stop at `BOS` or after 16 tokens.

Q37. What happens as $T \to 0$ and $T \to \infty$?

A. Greedy arg-max; uniform over the vocabulary.

Q38. Is "karia" a hallucination?

A. Yes, in exactly the sense the post means: a plausible sample that is not a real training example, produced by a model with no concept of truth.

Q39. Why is the graph still built during inference, and is that necessary?

A. Because every op creates `Values`; it is not necessary (no backward is run) and production code detaches.

23 Scaling up

Q40. Name the component-by-component changes from microgpt to a production LLM.

A. Data (trillions of tokens, curated); tokenizer (BPE); autograd (tensors, GPUs, fused kernels); architecture (bigger, RoPE, GQA, gated MLP, MoE); training (large batches, mixed precision); optimisation (tuned AdamW, schedules, scaling laws); post-training (SFT, RL); inference (batching, paging, speculation, quantisation).

Q41. What does SFT change algorithmically?

A. Nothing; the documents become conversations.

Q42. In RL post-training, where do the "documents" come from?

A. From the model's own samples, scored and fed back.

Q43. What rule of thumb does Chinchilla give?

A. Roughly 20 training tokens per parameter for compute-optimal training.

Q44. Which three knobs improve the names, according to the FAQ?

A. More steps, a bigger model, more data.

Part IV

Annotated bibliography

Entries are grouped by theme. A one-line note after each group says why it is here. Full details, including arXiv identifiers, are in the entries.

Karpathy: microgpt and its lineage

The primary source and the projects it distils. Read the post first, then micrograd, makemore, nanoGPT, nanochat.

Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks*. URL: https://karpathy.github.io/2015/05/21/rnn-effectiveness/ (visited on 10/11/2026).

Karpathy, A. (2020). *micrograd: a tiny scalar-valued autograd engine and a neural net library on top of it*. URL: https://github.com/karpathy/micrograd (visited on 10/11/2026).

Karpathy, A. (2022a). *makemore: an autoregressive character-level language model for making more things*. Source of the `names.txt` dataset used by microgpt. URL: https://github.com/karpathy/makemore (visited on 10/11/2026).

Karpathy, A. (2022b). *nanoGPT: the simplest, fastest repository for training/finetuning medium-sized GPTs*. URL: https://github.com/karpathy/nanoGPT (visited on 10/11/2026).

Karpathy, A. (2022c). *Neural Networks: Zero to Hero*. Video lecture series: micrograd, makemore, GPT, tokenizer. URL: https://karpathy.ai/zero-to-hero.html (visited on 10/11/2026).

Karpathy, A. (2022d). *The spelled-out intro to neural networks and backpropagation: building micrograd*. Video lecture, approx. 2.5 hours. URL: https://www.youtube.com/watch?v=VMj-3S1tku0 (visited on 10/11/2026).

Karpathy, A. (2023). *Let's build GPT: from scratch, in code, spelled out*. URL: https://www.youtube.com/watch?v=kCc8FmEb1nY (visited on 10/11/2026).

Karpathy, A. (2024a). *Let's build the GPT Tokenizer*. URL: https://www.youtube.com/watch?v=zduSFxRajkE (visited on 10/11/2026).

Karpathy, A. (2024b). *llm.c: LLM training in simple, raw C/CUDA*. URL: https://github.com/karpathy/llm.c (visited on 10/11/2026).

Karpathy, A. (2025). *nanochat: the best ChatGPT that $100 can buy*. Full pretraining, SFT, RL and serving pipeline in one small repo. URL: https://github.com/karpathy/nanochat (visited on 10/11/2026).

Karpathy, A. (2026a). *build_microgpt.py: builds up microgpt.py piece by piece*. GitHub gist; the Revisions tab holds train0.py through train5.py. URL: https://gist.github.com/karpathy/561ac2de12a47cc06a23691e1be9543a (visited on 10/11/2026).

Karpathy, A. (Feb. 2026b). *microgpt*. Blog post; the primary source for this course. URL: https://karpathy.github.io/2026/02/12/microgpt/ (visited on 10/11/2026).

Karpathy, A. (2026c). *microgpt Google Colab notebook*. URL: https://colab.research.google.com/drive/1vyN5zo6rqUp_dYNbT4Yrco66zuWCZKoN (visited on 10/11/2026).

Karpathy, A. (2026d). *microgpt.py: the most atomic way to train and run inference for a GPT in pure, dependency-free Python*. GitHub gist, 200 lines. URL: https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95 (visited on 10/11/2026).

OpenAI (2022). *tiktoken: a fast BPE tokeniser for use with OpenAI's models*. URL: https://github.com/openai/tiktoken (visited on 10/11/2026).

Language modelling: foundations

Where next-symbol prediction, entropy and neural language models come from.

Bahdanau, D., K. Cho, and Y. Bengio (2015). "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations (ICLR)*. arXiv:1409.0473.

Bengio, Y., R. Ducharme, P. Vincent, and C. Jauvin (2003). "A Neural Probabilistic Language Model". In: *Journal of Machine Learning Research* 3, pp. 1137–1155.

Elman, J. L. (1990). "Finding Structure in Time". In: *Cognitive Science* 14.2, pp. 179–211.

Graves, A. (2013). "Generating Sequences With Recurrent Neural Networks". In: *arXiv preprint arXiv:1308.0850*.

Hochreiter, S. and J. Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Mikolov, T., M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur (2010). "Recurrent Neural Network Based Language Model". In: *Proceedings of Interspeech*, pp. 1045–1048.
Shannon, C. E. (1948). "A Mathematical Theory of Communication". In: *Bell System Technical Journal* 27.3, pp. 379–423.
Shannon, C. E. (1951). "Prediction and Entropy of Printed English". In: *Bell System Technical Journal* 30.1, pp. 50–64.
Sutskever, I., J. Martens, and G. Hinton (2011). "Generating Text with Recurrent Neural Networks". In: *Proceedings of the 28th International Conference on Machine Learning (ICML)*.

Tokenization

From characters to byte-pair encoding.

Gage, P. (1994). "A New Algorithm for Data Compression". In: *C Users Journal* 12.2, pp. 23–38.
Kudo, T. and J. Richardson (2018). "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing". In: *Proceedings of EMNLP: System Demonstrations*. arXiv:1808.06226.
Sennrich, R., B. Haddow, and A. Birch (2016). "Neural Machine Translation of Rare Words with Subword Units". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:1508.07909, pp. 1715–1725.

Automatic differentiation and backpropagation

The mathematics and history behind `Value.backward()`.

Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018). "Automatic Differentiation in Machine Learning: a Survey". In: *Journal of Machine Learning Research* 18.153, pp. 1–43.
Griewank, A. and A. Walther (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd ed. SIAM.
Linnainmaa, S. (1976). "Taylor Expansion of the Accumulated Rounding Error". In: *BIT Numerical Mathematics* 16.2. The earliest published form of reverse-mode automatic differentiation, pp. 146–160.
Paszke, A. et al. (2017). "Automatic Differentiation in PyTorch". In: *NIPS 2017 Autodiff Workshop*.
Paszke, A. et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*.
Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). "Learning Representations by Back-Propagating Errors". In: *Nature* 323, pp. 533–536.

Transformer architecture

Attention, normalisation, residuals, activations, position encodings, and the modern substitutions (RoPE, GQA, MoE).

Ainslie, J., J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of EMNLP*. arXiv:2305.13245.
Alammar, J. (2018). *The Illustrated Transformer*. URL: https://jalammar.github.io/illustrated-transformer/ (visited on 10/11/2026).
Ba, J. L., J. R. Kiros, and G. E. Hinton (2016). "Layer Normalization". In: *arXiv preprint arXiv:1607.06450*.
Brown, T. B. et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. arXiv:2005.14165.
DeepSeek-AI (2024). "DeepSeek-V3 Technical Report". In: *arXiv preprint arXiv:2412.19437*. Large-scale MoE, multi-head latent attention, FP8 training.
Elhage, N. et al. (2021). *A Mathematical Framework for Transformer Circuits*. The "residual stream" view of the Transformer. URL: https://transformer-circuits.pub/2021/framework/index.html (visited on 10/11/2026).
Fedus, W., B. Zoph, and N. Shazeer (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23.120, pp. 1–39.
Glorot, X., A. Bordes, and Y. Bengio (2011). "Deep Sparse Rectifier Neural Networks". In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*.
He, K., X. Zhang, S. Ren, and J. Sun (2016). "Deep Residual Learning for Image Recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1512.03385, pp. 770–778.
Hendrycks, D. and K. Gimpel (2016). "Gaussian Error Linear Units (GELUs)". In: *arXiv preprint arXiv:1606.08415*.
Nair, V. and G. E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*.

Phuong, M. and M. Hutter (2022). "Formal Algorithms for Transformers". In: *arXiv preprint arXiv:2207.09238*. Pseudocode for every Transformer variant; an excellent companion to the code.

Pope, R. et al. (2022). "Efficiently Scaling Transformer Inference". In: *arXiv preprint arXiv:2211.05102*. Careful accounting of KV-cache memory and bandwidth.

Press, O. and L. Wolf (2017). "Using the Output Embedding to Improve Language Models". In: *Proceedings of EACL*. arXiv:1608.05859; microgpt deliberately does *not* tie wte and lm_head.

Radford, A., K. Narasimhan, T. Salimans, and I. Sutskever (2018). *Improving Language Understanding by Generative Pre-Training*. Tech. rep. OpenAI.

Radford, A., J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever (2019). *Language Models are Unsupervised Multitask Learners*. Tech. rep. The GPT-2 report; microgpt follows this architecture. OpenAI.

Shazeer, N. (2019). "Fast Transformer Decoding: One Write-Head is All You Need". In: *arXiv preprint arXiv:1911.02150*. Multi-Query Attention.

Shazeer, N. (2020). "GLU Variants Improve Transformer". In: *arXiv preprint arXiv:2002.05202*.

Shazeer, N., A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations (ICLR)*. arXiv:1701.06538.

So, D. R., W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le (2021). "Primer: Searching for Efficient Transformers for Language Modeling". In: *arXiv preprint arXiv:2109.08668*. Introduces squared ReLU in the Transformer MLP.

Su, J., Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *arXiv preprint arXiv:2104.09864*.

Touvron, H. et al. (2023). "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971*. Popularised the RMSNorm + SwiGLU + RoPE recipe.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NIPS)*. arXiv:1706.03762.

Xiong, R. et al. (2020). "On Layer Normalization in the Transformer Architecture". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. arXiv:2002.04745; the Pre-LN arrangement used by GPT-2 and microgpt.

Zhang, B. and R. Sennrich (2019). "Root Mean Square Layer Normalization". In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. arXiv:1910.07467.

Optimisation and scaling

SGD, momentum, RMSProp, Adam, schedules, initialisation, precision, scaling laws.

Glorot, X. and Y. Bengio (2010). "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*.

Hoffmann, J. et al. (2022). "Training Compute-Optimal Large Language Models". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.15556; the "Chinchilla" scaling laws.

Kaplan, J. et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*.

Kingma, D. P. and J. Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

Loshchilov, I. and F. Hutter (2017). "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *International Conference on Learning Representations (ICLR)*. arXiv:1608.03983; cosine learning-rate schedules.

Loshchilov, I. and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)*. arXiv:1711.05101; AdamW.

Micikevicius, P. et al. (2018). "Mixed Precision Training". In: *International Conference on Learning Representations (ICLR)*. arXiv:1710.03740.

Polyak, B. T. (1964). "Some Methods of Speeding Up the Convergence of Iteration Methods". In: *USSR Computational Mathematics and Mathematical Physics* 4.5, pp. 1–17.

Robbins, H. and S. Monro (1951). "A Stochastic Approximation Method". In: *The Annals of Mathematical Statistics* 22.3, pp. 400–407.

Tieleman, T. and G. Hinton (2012). *Lecture 6.5 – RMSProp: Divide the Gradient by a Running Average of its Recent Magnitude*. COURSERA: Neural Networks for Machine Learning.

Loss, information theory and textbooks

Cross-entropy, entropy, perplexity and general references.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
Cover, T. M. and J. A. Thomas (2006). *Elements of Information Theory*. 2nd ed. Wiley.
Goodfellow, I., Y. Bengio, and A. Courville (2016). *Deep Learning*. Free online at https://www.deeplearningbook.org. MIT Press.
Jurafsky, D. and J. H. Martin (2025). *Speech and Language Processing*. 3rd ed. draft. https://web.stanford.edu/~jurafsky/slp3/; chapters on n-grams, neural LMs, Transformers. Online.
Prince, S. J. D. (2023). *Understanding Deep Learning*. Free online at https://udlbook.github.io/udlbook/. MIT Press.

Sampling and decoding

Temperature and its companions.

Ackley, D. H., G. E. Hinton, and T. J. Sejnowski (1985). "A Learning Algorithm for Boltzmann Machines". In: *Cognitive Science* 9.1. Origin of the "temperature" softmax in neural networks, pp. 147–169.
Fan, A., M. Lewis, and Y. Dauphin (2018). "Hierarchical Neural Story Generation". In: *Proceedings of ACL*. arXiv:1805.04833; top-k sampling.
Hinton, G., O. Vinyals, and J. Dean (2015). "Distilling the Knowledge in a Neural Network". In: *arXiv preprint arXiv:1503.02531*. Clear discussion of softmax temperature.
Holtzman, A., J. Buys, L. Du, M. Forbes, and Y. Choi (2020). "The Curious Case of Neural Text Degeneration". In: *International Conference on Learning Representations (ICLR)*. arXiv:1904.09751; nucleus (top-p) sampling.

Post-training

SFT, RLHF, DPO, GRPO: from document completer to assistant.

Christiano, P. F., J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei (2017). "Deep Reinforcement Learning from Human Preferences". In: *Advances in Neural Information Processing Systems 30 (NIPS)*. arXiv:1706.03741.
Ouyang, L. et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.02155; InstructGPT: SFT + RLHF.
Rafailov, R., A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems 36 (NeurIPS)*. arXiv:2305.18290.
Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347*.
Shao, Z. et al. (2024). "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300*. Introduces GRPO.

Inference systems

Making the sampling loop fast.

Chen, C., S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper (2023). "Accelerating Large Language Model Decoding with Speculative Sampling". In: *arXiv preprint arXiv:2302.01318*.
Dao, T. (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations (ICLR)*. arXiv:2307.08691.
Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2205.14135.
Dettmers, T., M. Lewis, Y. Belkada, and L. Zettlemoyer (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2208.07339.
Frantar, E., S. Ashkboos, T. Hoefler, and D. Alistarh (2023). "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*. arXiv:2210.17323.
Kwon, W. et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. arXiv:2309.06180; vLLM.
Leviathan, Y., M. Kalman, and Y. Matias (2023). "Fast Inference from Transformers via Speculative Decoding". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. arXiv:2211.17192.

Yu, G.-I., J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun (2022). “Orca: A Distributed Serving System for Transformer-Based Generative Models”. In: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Continuous (iteration-level) batching.

Data

Corpora and curation.

Gao, L. et al. (2020). “The Pile: An 800GB Dataset of Diverse Text for Language Modeling”. In: *arXiv preprint arXiv:2101.00027*.

Lee, K., D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini (2022). “Deduplicating Training Data Makes Language Models Better”. In: *Proceedings of ACL*. arXiv:2107.06499.

Penedo, G., H. Kydlíček, L. B. Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf (2024). “The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale”. In: *arXiv preprint arXiv:2406.17557*.

Raffel, C. et al. (2020). “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140. Introduces the C4 web corpus, pp. 1–67.